A7 — Individual Submission

Student Name: Phyo Sithu
Student ID: 6731503082
Project / Capability Name: Support Inquiry Router — grounded customer inquiry classification
A6 Project Reference: `a6-customer-inquiry/` (A6 report: `assignment8_submission.md`, PDF: `A6_6731503082.pdf`)
Implementation Method: Jev Playground (keyless evaluator) with TypeScript integration

1. Grounding Context

Knowledge Source:
A small, synthetic support-routing policy base created for this capability. It makes the rules that were implicit in the A6 queue definitions explicit: who owns each issue, how multi-intent messages are prioritised, and when a human must review. It contains no customer data and is not copied from a real company; it is an internal policy artifact for the evaluation.

Knowledge Records:
Seven records in `a6-customer-inquiry/knowledge/knowledge-base.json`:

ID	Title	Queue	Purpose
KB-01	Billing queue ownership	billing	Charges, refunds, invoices, subscriptions; charge beats delivery/login mentions
KB-02	Technical queue ownership	technical	Errors, checkout/API failures, outages
KB-03	Account queue ownership	account	Login, 2FA, lockout, compromise; login is secondary when checking an order
KB-04	Shipping queue ownership	shipping	Delivery, tracking, damage, returns, replacements
KB-05	General triage and pre-sales	other	Discounts, availability, partnerships, press, feedback
KB-06	Multi-intent routing priority	policy	Security > money > delivery > technical > information
KB-07	Urgency and human-review policy	policy	Critical/high/normal/low rules and mandatory review triggers

Each record has a stable ID, title, queue, keyword list, and a short policy paragraph. The full file is linked in Section 8.

Retrieval / Selection Method:
Lexical retrieval, not a vector database. The message is lowercased and whitespace-normalised. Every record is scored by the policy keywords it contains: a matched multi-word phrase adds 2, a single word adds 1. KB-06 (multi-intent priority) and KB-07 (urgency/review) are **always supplied** because they govern every decision; the top three queue records by score are added. Ties are broken by record ID, so the selection is reproducible. The selected records are rendered as authoritative policy lines and inserted into the Jev state before the customer message.

Code: `a6-customer-inquiry/src/grounding.ts`. The evaluator records the selected IDs and matched keywords for every run in `eval/results-improved.json`.

```
export function retrieveKnowledge(text: string, options = {}): GroundingResult {
  const topK = options.topK ?? 3;
  const message = normalise(text);
  const alwaysOn = records.filter((r) => r.alwaysOn);          // KB-06, KB-07
  const ranked = records
    .filter((r) => !r.alwaysOn)
    .map((r) => scoreRecord(r, message))                       // phrase = 2, word = 1
    .filter((entry) => entry.score > 0)
    .sort((a, b) => b.score - a.score || a.record.id.localeCompare(b.record.id))
    .slice(0, topK);
  return { selected: [...alwaysOn, ...ranked], context: /* [ID] title: content */ };
}
```

Context Supplied to Model:
Actual improved state for T09 (`eval/results-improved.json` , run 1), with the two queue records shortened for readability:

```
Relevant platform policy (retrieved, authoritative):
- [KB-06] Multi-intent routing priority: When a message contains several issues, choose exactly one queue using this priority order: (1) account security or compromise, (2) money at risk such as a duplicate or unauthorized charge or a refund, (3) physical delivery, return, or replacement, (4) technical failure, (5) information or pre-sales. A login problem that only prevents the customer from checking an order or payment is not a security issue and must not outrank the money issue. The other issues are handled as secondary tasks.
- [KB-07] Urgency and human review policy: Urgency: critical for account compromise, legal threat or chargeback, safety issue, or a widespread outage; high for a blocked workflow, money at risk, repeated failure, or a premium or business customer; normal for a routine request; low for informational or pre-sales questions. Human review is required for security incidents, legal threats, refunds above USD 100, and any message with three or more distinct issues.
- [KB-03] Account queue ownership: ... If login is mentioned only as a way to check an order or payment, do not choose account; route the underlying money or delivery issue instead.
- [KB-01] Billing queue ownership: ... A charge or refund dispute belongs to billing even when the customer also mentions a delivery problem or a login problem. ...

Channel: email
Customer tier: standard
Customer message:
I was charged twice this month, and I can't log in to check my invoices.
```

This context is useful because it contains decisions the A6 model could not infer from a single message: the **priority between queues** when several issues appear, and the **platform's review policy**. In the baseline, the same message made Jev choose `account` with 0.40–0.57 confidence and fall back. With the policy, it chose `billing` with confidence 1.00. The context does not include the expected answer or any test-specific hint.

2. Evaluation Dataset

Ten cases fixed before the measured runs. `T07` / `T08` intentionally force a model failure to evaluate the fallback. Primary pass = category + human-review flag + source; strict pass also checks urgency.

ID	Case Type	Input	Expected Output	Expected Review/Fallback	Reason
T01	Normal	"I was charged twice for my September subscription. Please refund the duplicate charge to my card."	billing; u3; no human; jev	None; auto-route	KB-01: billing owns duplicate charges/refunds. KB-07: money at risk is high urgency; no review trigger.
T02	Normal	"Checkout fails with a 500 error every time I click Pay. I have tried three browsers, and customers cannot complete any orders right now."	technical; u4; human; jev	Human review forced by critical urgency	KB-02: checkout failure/outage is technical. KB-07: widespread outage is critical.
T03	Normal	"I can't log in. The password reset email never arrives, so I'm locked out of my account."	account; u3; no human; jev	None; auto-route	KB-03: login/reset is account. Blocked workflow is high, but no security/legal trigger.
T04	Edge	"Do you offer a student discount on the annual plan? I'm deciding whether to buy."	other; u1; human; jev	Human review because category is `other`	KB-05: pre-sales discount is general triage, not billing (no completed charge). A6 policy sends `other` to review.
T05	Edge	"My lamp arrived damaged. I want a replacement, not a refund."	shipping; u2; no human; jev	None; auto-route	KB-04: damaged items/replacements are shipping. The customer explicitly declines a refund, so the billing exception does not apply.

ID	Case Type	Input	Expected Output	Expected Review/Fallback	Reason
T06	Edge	"Someone accessed my account, changed the email address, and locked me out. I have contacted my lawyer."	account; u4; human; jev	Human review required	KB-03: compromise is account. KB-06: security outranks all. KB-07: compromise + legal threat is critical.
T07	Fallback	"I was charged twice for my subscription and want a refund."	billing; any urgency; human; fallback	Forced HTTP 503; deterministic fallback + review	Degraded path must not lose the ticket or auto-route.
T08	Fallback	"Where is my parcel? The tracking has not updated for a week."	shipping; any urgency; human; fallback	Forced timeout; deterministic fallback + review	A slow model must not block the queue; fallback classifies from delivery terms.
T09	Difficult	"I was charged twice this month, and I can't log in to check my invoices."	billing; u3; no human; jev	None; auto-route	KB-06: money outranks a login symptom. KB-03: login is only how the customer checks invoices. Two issues do not trigger mandatory review.
T10	Difficult	"My order arrived damaged, I was charged twice, and now I can't log in to request a return."	billing; u3; human; jev	Human review because three issues	KB-06: money outranks delivery/access. KB-07: three distinct issues require review; no security/legal/outage, so high not critical.

Dataset: `eval-cases.json` (workspace root) and `a6-customer-inquiry/eval/eval-cases.json`.

3. Baseline Evaluation

Baseline = the original A6 request (state + questions, **no retrieved policy**), same model, same thresholds, same fallback. The decision shown is run 1; all three runs produced the same label/source/review pattern. Latency is the median of three runs.

ID	Expected Output	Actual Output	Pass/Fail	Latency (ms)
T01	billing; u3; no human; jev	billing; u3; no human; jev (conf 1.00)	Pass	956
T02	technical; u4; human; jev	technical; u4; human; jev (conf 0.99)	Pass	1449
T03	account; u3; no human; jev	account; u3; no human; jev (conf 1.00)	Pass	759
T04	other; u1; human; jev	**billing**; u1; human; **fallback** (conf n/a)	Fail	654
T05	shipping; u2; no human; jev	shipping; u2; no human; jev (conf 1.00)	Pass	626
T06	account; u4; human; jev	account; u4; human; jev (conf 1.00)	Pass	833
T07	billing; human; fallback	billing; u1; human; fallback (conf n/a)	Pass	0
T08	shipping; human; fallback	shipping; u3; human; fallback (conf n/a)	Pass	0
T09	billing; u3; no human; jev	billing; u3; **human; fallback** (conf n/a)	Fail	755
T10	billing; u3; human; jev	**shipping**; u3; human; **fallback** (conf n/a)	Fail	1067

Baseline Accuracy: 7 / 10 = 70%

p95 Latency: 2357 ms over 24 measured model-path requests (nearest-rank). Forced-failure T07/T08 are excluded.

Cost per Task: $0.0000241 estimated. Measured average: 573 input tokens per task, 17,187 tokens over 30 requests. At USD 0.042 per 1M input tokens (output free), that is $0.024 per 1,000 tasks.

Measurement Method:
Three repetitions per case per version (30 requests, 24 model calls because T07/T08 fail before the network). A warm-up call was excluded from measurement. Latency is wall-clock around `classifyInquiry`; retries and forced-failure samples are excluded. Model latency is Jev's upstream `latencyMs`. `p95` uses the nearest-rank method (`ceil(0.95 ×`

`24) = 23rd` value). Token counts are the upstream `usage.inputTokens` recorded in `eval/results-baseline.json`. The cost is an **estimate**, not an invoice: the keyless Playground does not bill this account. Accuracy is case-level: a case passes only if all three runs match category, review flag, and source.

Case-level failures came from the confidence gate, not random labels:

```
T04: model → other, confidence 0.40/0.42/0.44 (< 0.60) → fallback → billing (wrong)
T09: model → account, confidence 0.40/0.57/0.57 (< 0.60) → fallback → billing (right category,
     wrong review flag and source)
T10: model → account, confidence 0.47/0.42/0.49 (< 0.60) → fallback → shipping (wrong)
```

4. One Controlled Improvement

Selected Improvement:
Add retrieved policy grounding to the Jev state. This is the only change: the model, the three questions, the 0.60/0.75 confidence thresholds, validation, urgency mapping, and fallback rules are unchanged. The baseline sends the A6 state; the improved version prepends KB-06/KB-07 plus the top three lexical matches.

Reason for Choosing It:
The baseline's three failures all had the same shape: the model returned a plausible label with confidence below 0.60, and the fallback either changed the label (T04, T10) or degraded a correct label into a human-review fallback (T09). The model did not know the platform's queue-ownership policy or the priority order for multi-intent messages. This was already visible in A6, where a mixed billing/shipping/login message was sent to `account` at 0.40 confidence. Supplying the actual policy addresses the observed problem without changing the model or inventing a new classifier.

Before Configuration:
`src/jev.ts` → `buildDecisionRequest(input)` → `buildState` with no context:

```
export function buildState(input) {
  return [
    `Channel: ${input.channel}`,
    `Customer tier: ${input.customerTier}`,
    input.orderId ? `Related order: ${input.orderId}` : null,
    "Customer message:",
    input.message,
  ].filter((line) => line !== null).join("\n");
}
```

After Configuration:
`src/config.ts` accepts `context`, and `src/grounding.ts` retrieves the policy:

```
const grounding = retrieveKnowledge(input.message);   // KB-06, KB-07 + top 3
const decision = await runJevDecision({ ...input, context: grounding.context });
```

```
// src/config.ts (excerpt)
const lines = [
  input.context?.length
    ? [
        "Relevant platform policy (retrieved, authoritative):",
        ...input.context.map((record) => `- ${record}`),
        "",
      ].join("\n")
    : null,
  `Channel: ${input.channel}`,
  `Customer tier: ${input.customerTier}`,
  input.orderId ? `Related order: ${input.orderId}` : null,
  "Customer message:",
  input.message,
];
```

`retrieveKnowledge` returns the always-on priority/urgency records plus the three queue records with the highest keyword score. The selected IDs for every case are in `eval/summary.md` (retrieval evidence) and `eval/results-improved.json`. The HTTP server uses the same pairing (`src/server.ts` → `groundedDecide`), so the running capability is the improved one; the A6 tests inject a decision function without context to keep the baseline reproducible.

5. After-Improvement Evaluation

Same 10 cases, same model, same thresholds and fallback; only the retrieved context changed. Three runs per case; all decisions were stable across runs.

ID	Expected Output	Actual Output	Pass/Fail	Latency (ms)
T01	billing; u3; no human; jev	billing; u3; no human; jev (conf 1.00)	Pass	831
T02	technical; u4; human; jev	technical; u4; human; jev (conf 1.00)	Pass	806
T03	account; u3; no human; jev	account; u3; no human; jev (conf 1.00)	Pass	910
T04	other; u1; human; jev	other; u1; human; jev (conf 1.00)	Pass	981
T05	shipping; u2; no human; jev	shipping; u2; no human; jev (conf 1.00)	Pass	738
T06	account; u4; human; jev	account; u4; human; jev (conf 1.00)	Pass	877
T07	billing; human; fallback	billing; u1; human; fallback (conf n/a)	Pass	0
T08	shipping; human; fallback	shipping; u3; human; fallback (conf n/a)	Pass	0
T09	billing; u3; no human; jev	billing; u3; no human; jev (conf 1.00)	Pass	892
T10	billing; u3; human; jev	billing; u3; human; jev (conf 0.96)	Pass	877

After Accuracy: 10 / 10 = 100%. Strict accuracy (including urgency) is also 10/10.

p95 Latency: 1929 ms over 24 measured model-path requests. Model-call p95 is 735 ms (median 597 ms).

Cost per Task: $0.0000357 estimated. Measured average: 849 input tokens per task (25,482 tokens over 30 requests). At USD 0.042 per 1M input tokens, that is $0.036 per 1,000 tasks.

6. Before / After Comparison

Metric	Before	After	Change
Accuracy (case-level, all 3 runs)	70%	100%	+30 pp
Accuracy (strict, includes urgency)	70%	100%	+30 pp
p95 latency, measured requests	2357 ms	1929 ms	−428 ms

Metric	Before	After	Change
p95 latency, model calls only	616 ms	735 ms	+119 ms (+19%)
Avg input tokens / task	573	849	+277 (+48%)
Est. cost / task	$0.0000241	$0.0000357	+48%
Est. cost / 1,000 tasks	$0.0241	$0.0357	+$0.0116

The measured-request p95 is lower in this sample, but that is **not** evidence that grounding makes the service faster: it was driven by a 4.8 s baseline outlier in T09. The fair latency comparison is model-call p95, which increased by 119 ms because the prompt grew by ~277 tokens. The median model latency also rose (449 ms → 597 ms).

Cases That Improved:

- **T04** — The baseline model already chose `other`, but at confidence 0.40–0.44, so the platform rejected it and the fallback picked `billing`. Grounding raised confidence to 1.00 and preserved `other`, the correct pre-sales queue. The improvement fixed trust, not the semantic guess.
- **T09** — The baseline model chose `account` (0.40–0.57); fallback produced the right category `billing` but with `needsHuman: true` and `source: fallback`. Grounding made it `billing` with confidence 1.00, no unnecessary human review, and `source: jev`. This case shows why evaluating only category accuracy would have hidden a degraded decision.
- **T10** — The baseline model chose `account` (0.42–0.49) and the fallback chose `shipping`; both are wrong. Grounding produced `billing` with 0.94–0.96 confidence and correctly forced human review for three issues.

Cases That Became Worse:
None. The five cases that already passed kept the same decision and source. T01–T03, T05–T06 remained correct; the two forced-fallback cases are unaffected by design.

Trade-offs:
Accuracy improved by 30 percentage points with no regression, but the prompt grew by 48%, cost per task rose by 48%, and model-call p95 rose by 19%. Grounding also adds a new failure surface: the knowledge base must be versioned and maintained, retrieval can select an irrelevant or contradictory record, and stale policy will be followed confidently. The all-request p95 difference should be treated as noise, not a benefit.

Conclusion:
The improvement is worthwhile for this policy-heavy routing problem: it eliminated all three baseline failures, made every decision stable across three runs, and removed unnecessary fallbacks. It is not yet production-ready. The evaluation has only 10 cases from one author, one model version, and a synthetic policy. A production rollout should first run in shadow mode against human triage with a much larger, independently labelled set, and should monitor fallback rate, human overrides, and cost per resolved ticket.

7. Engineering Reflection

What I learned:
Systematic evaluation exposed failures that the A6 demo hid. A6 reported 8/8 because its cases were mostly clean single-intent messages; the 10-case set with edge and ambiguous inputs found three cases where the model was semantically close but its confidence fell below the gate. It also taught me to measure the **source** and **review flag**, not just the category: T09 would look correct if I only compared labels, yet it was a degraded fallback with an unnecessary human review. Three repetitions per case showed that the baseline failures were stable, not random, which makes the improvement claim stronger. Finally, the cost/latency trade-off is real: better context is not free, and a 48% token increase is material at volume.

Why a few successful examples are insufficient:
A handful of successful examples only shows that the system can work; it does not show where it stops working. Clean examples share the same surface pattern as the prompt, while edge cases test policy boundaries, multi-intent priority, low-confidence behaviour, and failure handling. A6's 8/8 gave false confidence because it did not include pre-sales questions, charge-plus-login messages, or three-issue messages. One run per case would also have hidden instability. Expected results must be written before execution; otherwise it is easy to rationalise whatever the model produced.

What I would evaluate next:
Before production I would build a larger, stratified, independently labelled set (at least 100–200 inquiries) and report per-category precision/recall, a confusion matrix, and calibration (does 0.90 confidence actually mean 90% correct?). I would evaluate retrieval separately: recall of the right policy record, behaviour when no record is relevant, and resistance to contradictory or stale records. I would also test multilingual messages, typos and negation, prompt injection while grounded, and the human-review queue's precision. Operationally, I would measure p99 latency and cost per resolved ticket at realistic volume, monitor fallback and override rates by category, and version both the model and the policy base so a change can be rolled back. A shadow-mode trial against human triage is the next gate before any automatic routing.

8. Evidence and References

Supporting files:

File	Contents
`eval-cases.json`	The 10 evaluation cases with expected results and reasons
`a6-customer-inquiry/eval/results-baseline.json`	30 raw baseline runs (outputs, confidence, source, latency, tokens)
`a6-customer-inquiry/eval/results-improved.json`	30 raw improved runs + retrieved IDs and full prompts
`a6-customer-inquiry/eval/summary.md` / `eval-summary.md`	Generated per-case and before/after comparison
`a6-customer-inquiry/knowledge/knowledge-base.json`	Seven policy records
`a6-customer-inquiry/src/grounding.ts`	Retrieval and context formatting
`a6-customer-inquiry/src/config.ts`	Before/after state construction
`a6-customer-inquiry/eval/run-eval.ts`	Reproducible harness (`npm run eval`)
`a6-customer-inquiry/eval/run.log`	Execution log from the measured run

Reproduction: from `a6-customer-inquiry/` , run `EVAL_RUNS=3 EVAL_PACING_MS=7000 npm run eval` . The harness warms up, runs both versions, retries 429/5xx with backoff, and writes the JSON results and summary. The A6 capability is unchanged: `npm test` and `npm run demo:fallback` still pass.

Measured vs estimated: Labels, confidence, urgency, review flags, source, wall-clock latencies, upstream `latencyMs` , and input-token counts are measured. The USD cost is estimated from measured tokens at the published Jev rate of **USD 0.042 per 1M input tokens, output free**; the keyless Playground does not issue a bill. No production traffic or invoice data is used.

References:

- Assignment A7 brief (`assignment_9.md`) and A6 project (`a6-customer-inquiry/` , `assignment8_submission.md`).
- Jev Playground evaluator — https://jevplayground.com/; official TypeSafe endpoint `POST https://api.typesafe.ai/v1/systemone` .
- Jev public pricing (USD 0.042 per 1M input tokens, output free), as reported by Layer3 Labs, "Jev Pricing" — https://www.layer3labs.io/guides/jev-pricing — and eesel AI, "TypeSafe Jev pricing 2026" — https://www.eesel.ai/blog/typesafe-jev-pricing.
- Node.js, TypeScript, zod, and Hono documentation as used in A6.

AI assistance disclosure: An AI coding assistant (OpenCode) was used to scaffold the knowledge base, retrieval, evaluation harness, and to help draft this report. The policy records and expected results were reviewed and fixed before the measured runs; all numbers in this report come from the JSON evidence produced by `eval/run-eval.ts` on 2026-10-22/23 (UTC) and are reproducible with the command above. All test messages are synthetic; no API keys or private customer data are included.

9. Submission Checklist

- ✓ Same capability as Week 08
- ✓ At least 5 knowledge records (7)
- ✓ 10 evaluation cases with expected results
- ✓ Baseline results completed (3 runs per case)
- ✓ Exactly one improvement applied (retrieved policy context)
- ✓ After-improvement results completed (3 runs per case)
- ✓ Before/after comparison included
- ✓ Actual evidence and reflection attached